

2. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

data Tree a = E | Leaf a | Node (Tree a) (Tree a)

donde el recorrido *inorder* del árbol da el orden de los elementos de la secuencia.

Definir en Haskell de manera eficiente la función `intercalate :: Tree a → Tree (Tree a) → Tree a`, que dada una secuencia s y una secuencia de secuencias xs , inserta s entre los elementos de xs y aplana la secuencia resultado. Por ejemplo,

`intercalate (0,1) (<(2,3), (4,5,6), (8)>) = <2,3,0,1,4,5,6,0,1,8>`
`intercalate () (<(1,2), (3,4,5)>) = <1,2,3,4,5>`
`intercalate (0,1) () = ()`

Definir `intercalate` con profundidad en $O(h)$, donde h es la altura del árbol que recibe como segundo argumento la función.

2. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

data Tree a = Leaf a | Node Int (Tree a) (Tree a)

donde se guarda la longitud de la secuencia en los nodos y el recorrido *inorder* del árbol da el orden de los elementos de la secuencia.

a) Definir en Haskell de manera eficiente la función `mapReduceIndex :: (Int → a → b) → (b → b → b) → Tree a → b`, que dados una función $f :: \text{Int} \rightarrow a \rightarrow b$, un operador binario asociativo $op :: b \rightarrow b \rightarrow b$ y una secuencia s , computa el resultado de aplicar el operador op sobre los elementos de la secuencia que resulta de aplicar la función f a cada elemento de s usando como primer argumento su índice en s , en el orden dado por el recorrido *inorder*.

Por ejemplo,

`mapReduceIndex f op (x0, x1, x2) = ((f 0 x0) 'op' (f 1 x1)) 'op' (f 2 x2)`

Definir `mapReduceIndex` con profundidad en $O(h)$, donde h es la altura del árbol y f y op son de costo constante.

Plantee las recurrencias para el trabajo y la profundidad para el caso en que f y op son de costo constante. No hace falta resolverlas.

b) Usando la función `mapReduceIndex`, defina una función `sumEven :: Tree Int → Int` que compute la suma de los números en las posiciones pares de una secuencia.

Por ejemplo,

`sumEven (7, 1, 5, 7, 2) = 14`

b) Para implementar las operaciones del TAD `MultiDic` de manera eficiente en Haskell se utilizan árboles binarios de búsqueda, representados con el siguiente tipo de datos:

data MultiDic k v = E | N (MultiDic k v) (k, Tree v) (MultiDic k v)

donde los valores asociados a las claves se almacenan en árboles binarios que contienen información sobre la cantidad de nodos:

data Tree a = Empty | Leaf a | Node Int (Tree a) (Tree a)

Usando esta representación definir en Haskell de manera eficiente las funciones:

i. `isValue :: (Ord k, Eq v) ⇒ k → v → MultiDic k v → Bool`, del TAD.

ii. `toMap :: Ord k ⇒ MultiDic k v → Tree (k, Int, v)`, similar a la función `toMap` del TAD, pero que agrega a cada par de la forma (clave, valor) la posición del valor en la secuencia representada por el árbol.

Por ejemplo,

`toMap (<(1, (a, f, g)), (2, (m, a))>) = <(1, 0, a), (1, 1, f), (1, 2, g), (2, 0, m), (2, 1, a)>`

2. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

```
data Tree a = E | N Int (Tree a) a (Tree a)
```

donde se guarda la longitud de la secuencia en los nodos y el recorrido *inorder* del árbol da el orden de los elementos de la secuencia.

Definir en Haskell de manera eficiente la función `filterPrefix :: (a → Bool) → Tree a → Tree a`, que dado un predicado p y una secuencia s , computa el prefijo más largo de s para el cual todos sus elementos satisfacen p .

Por ejemplo,

```
filterPrefix odd (6,6,8,1,4,5) = ()
```

```
filterPrefix even (6,6,8,1,4,5) = (6,6,8)
```

Definir `filterPrefix` con profundidad en $O(h)$, donde h es la altura del árbol y p es de costo constante.

1. Suponiendo una representación de secuencias con árboles binarios, definidos con el siguiente tipo de datos:

```
data Tree a = E | L a | N Int (Tree a) (Tree a)
```

donde se almacenan los tamaños de los árboles en los nodos y el recorrido *inorder* del árbol da el orden de los elementos de la secuencia. definir en Haskell, de manera eficiente, paralelizando cuando sea posible, las siguientes operaciones sobre secuencias:

- a) La función `concat :: Tree (Tree a) → Tree a`, que concatena una secuencia de secuencias. Por ejemplo,

```
concat < <5,2,3>, <6>, <8,0> > = <5,2,3,6,8,0>
```

- b) La función `subsequence :: Tree a → Int → Int → Tree a`, que dada una secuencia s y dos enteros i y j , devuelve la subsecuencia de s que está entre los índices i y j . Por ejemplo,

```
subsequence < 1, 2, 3, 4, 5, 6 > 2 4 = < 2, 3, 4 > = < 3, 4, 5 > (iniziamos la seq en 0)
```